

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY

VIZIANAGARAM

VIZIANAGARAM – 535003

ANDHRA PRADESH

VOLUNTEER FINDER APP

By

MIDASALA VENKATA DINESH

(Regd. No. 24VV1A1242)

Under the supervision of

MRS. MADHUMITA CHANDA

Department of Information Technology

Viziangaram – 535003



JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY

VIZIANAGARAM

VIZIANAGARAM – 535003

ANDHRA PRADESH

Regd.No:24VV1A1242

This is to certify that this is a bonafide record of practical work done by MIDASALA VENKATA DINESH of III B.Tech I Semester Class in USER INTERFACEDESIGN USING FLUTTER Lab during the year 2025-26.

No.of Tasks Completed and Certified:

Lecture In-Charge **Head of The Department**

Date:

DEPARTMENT OF INFORMATION TECHNOLOGY
JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM

Website:www.jntugvcev.edu.in

Subject Name: USER INTERFACE DESIGN USING FLUTTER Subject Code: R233112ES01

Academic Year: 2026 **Regulation: R23**

COURSE OUTCOMES

NBA Subject Code	Course Outcomes	
R233112ES01	CO1	Learns to Implement Flutter Widgets and Layouts.
	CO2	Understands Responsive UI Design and with Navigation in Flutter.
	CO3	Knowledge on Widgets and customize widgets for specific UI elements, Themes
	CO4	Understand to include animation apart from fetching data.

CO-PO Mapping

Mapping of Course Outcomes (COs) with Program Outcomes (POs)

Course Outcomes		Program Outcomes (POs)														
		PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO 1	PSO 2	PSO 3
R233112ES01	CO1	3	2	3	2	3	1	1	1	2	2	1	2	3	3	2
	CO2	3	3	3	2	3	1	2	2	2	3	2	2	3	3	2
	CO3	3	2	3	2	3	2	1	1	2	2	1	2	3	2	2
	CO4	3	3	3	3	3	2	1	2	3	3	2	3	3	3	3

Enter correlation levels 1,2 and 3 as defined below:

1: Slight (Low) 2: Moderate (Medium) 3: Substantial (High) If there is no correlation, put "-"

Signature of the Course Instructor

Contents

1 Introduction 1

2 Project Structure 2

3 Firebase Installation 3

4 Installation of Flutter (with PATH Setup) 5

5 Installation of Dart SDK in Windows 9

6 Installing Visual Studio Code 10

7 Creating a New Flutter Project 11

8 Dart Basic Programming 13

9 Case Study on Quiz Leaderboard App 16

10 Exploring Flutter Widgets 17

11 Responsive UI Design 25

12 Navigation Between Screens 28

13 Stateful and Stateless Widgets 29

14 Custom Widgets and Styling 31

15 Forms and Validation 34

16 Animations 43

17 REST API Integration 47

18 Testing and Debugging 47

19 Work Flow Of Screens: 49

20 Conclusion 52

1 Introduction

Flutter is an open-source software development framework created by Google. It allows developers to build high-quality, natively compiled applications for mobile, web, and desktop from a single codebase.

Advantages of Flutter

- Cross-platform development with a single codebase.
- Hot reload for instant updates.
- Rich set of customizable widgets.
- High performance due to AOT compilation.
- Backed by Google and strong community support.

Dart Language

Dart is the programming language used by Flutter. It is object-oriented, class-based, and supports JIT and AOT compilation. Dart syntax is simple and similar to Java/C++.

Advantages of Dart

- Easy-to-learn syntax.
- Null safety to avoid runtime errors.
- Strong typing and async/await support.
- JIT for fast development, AOT for optimized production.

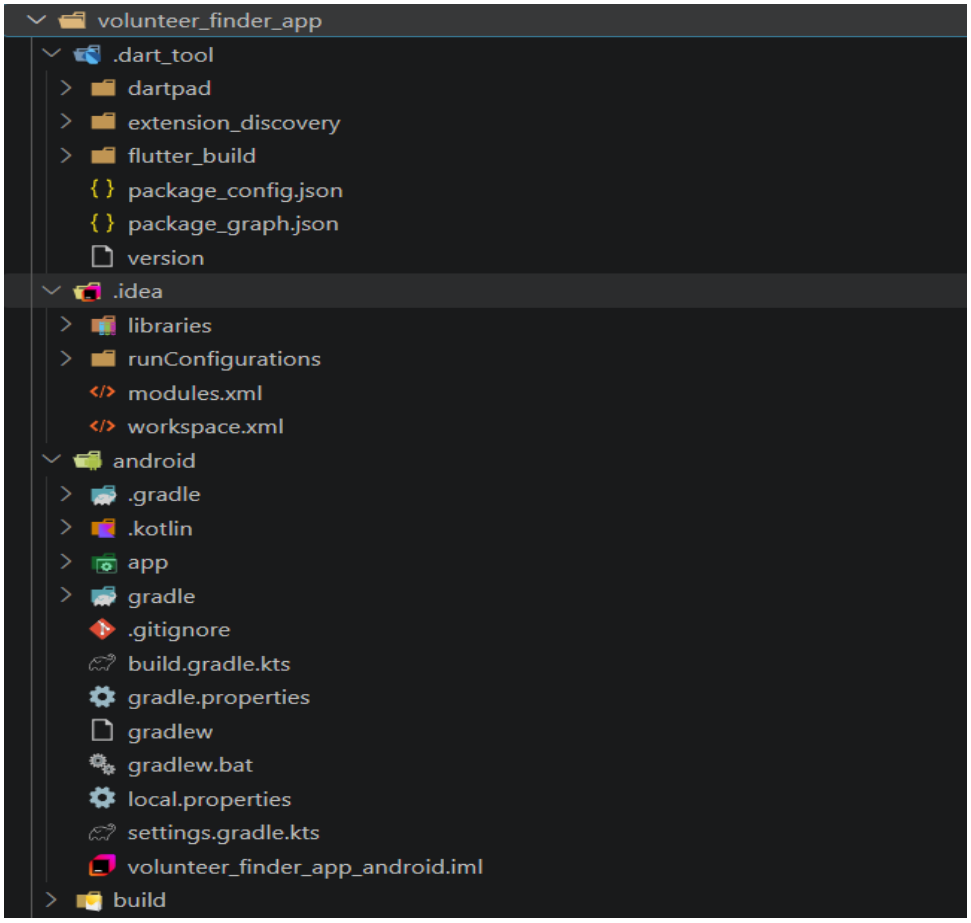
Key Features of Flutter

- **Cross-Platform Development:** One codebase can run on multiple platforms such as Android, iOS, Web, Windows, macOS, and Linux.
- **Hot Reload:** Instantly view code changes without restarting the app, improving developer productivity and speeding up testing.
- **Widget-Based Framework:** Everything in Flutter is a widget — layouts, buttons, text, and images, which makes UI creation simple and flexible.
- **High Performance:** Flutter apps are compiled to native ARM code, ensuring fast performance and smooth animations.
- **Custom UI Design:** Allows the creation of flexible and visually appealing interfaces using Material Design (for Android) or Cupertino (for iOS) widgets.

2 Project Structure

The **Quiz Leaderboard App** is a Flutter-based mobile application designed to create an interactive quiz experience for users. It allows them to register, log in, participate in quizzes, and track their scores in a global leaderboard. The app demonstrates real-world Flutter practices including navigation, widget composition, form validation, state management, and integration with backend APIs.

Below is the directory structure of the project:



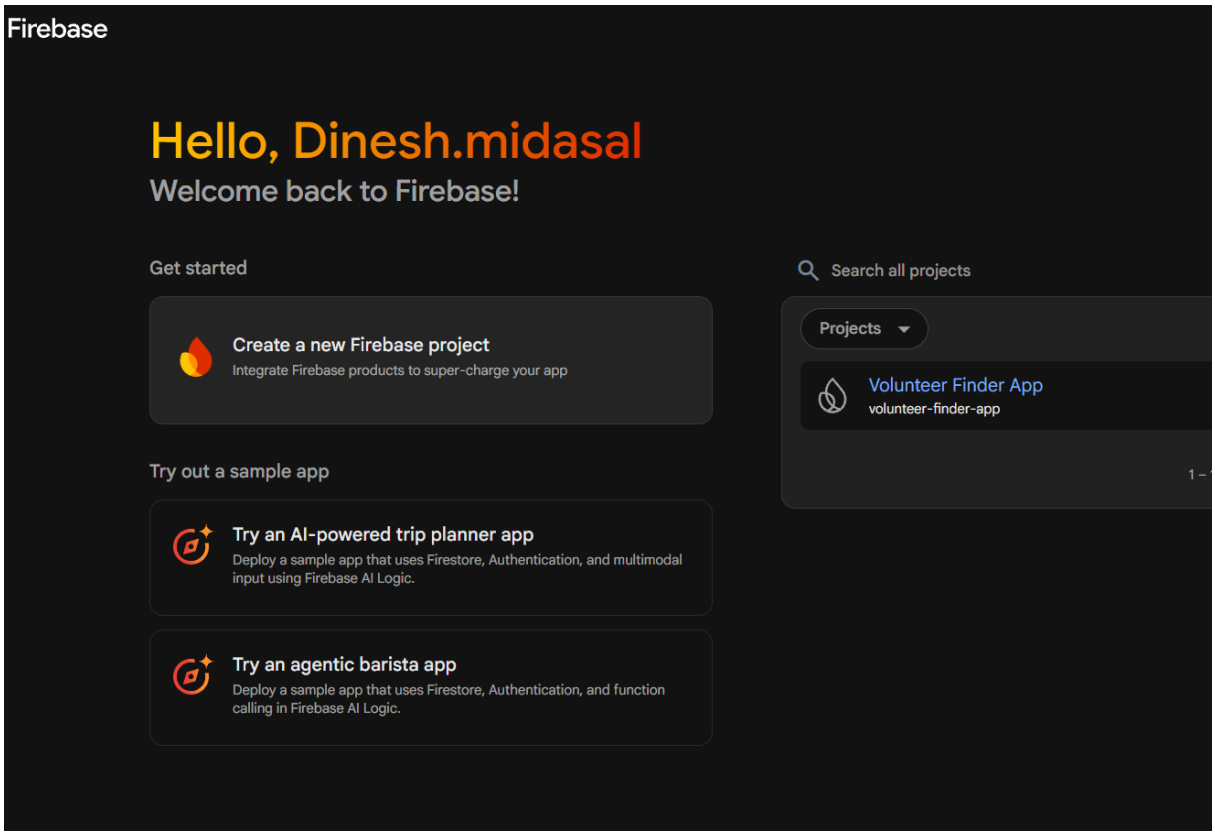
Description of Project Modules

- **lib/** – The core directory of the Flutter app containing all Dart source files, organized into subfolders for modularity and clarity.
- **data/** – Contains local data files used by the app. The file quiz_data.dart holds the static or mock data for quiz questions, answers, and scores used when the backend is unavailable.
- **models/** – Defines data structures that represent the core entities of the app. The quiz.dart model defines the structure of a quiz, including question text, options, and correct answers.
- **screens/** – Contains all the major UI screens of the application. Each screen represents a key feature:

3 Firebase Installation

Firebase is a platform developed by Google for creating mobile and web applications. It provides services like real-time databases, authentication, cloud storage, and more. The steps to install and configure Firebase for a Flutter project are as follows:

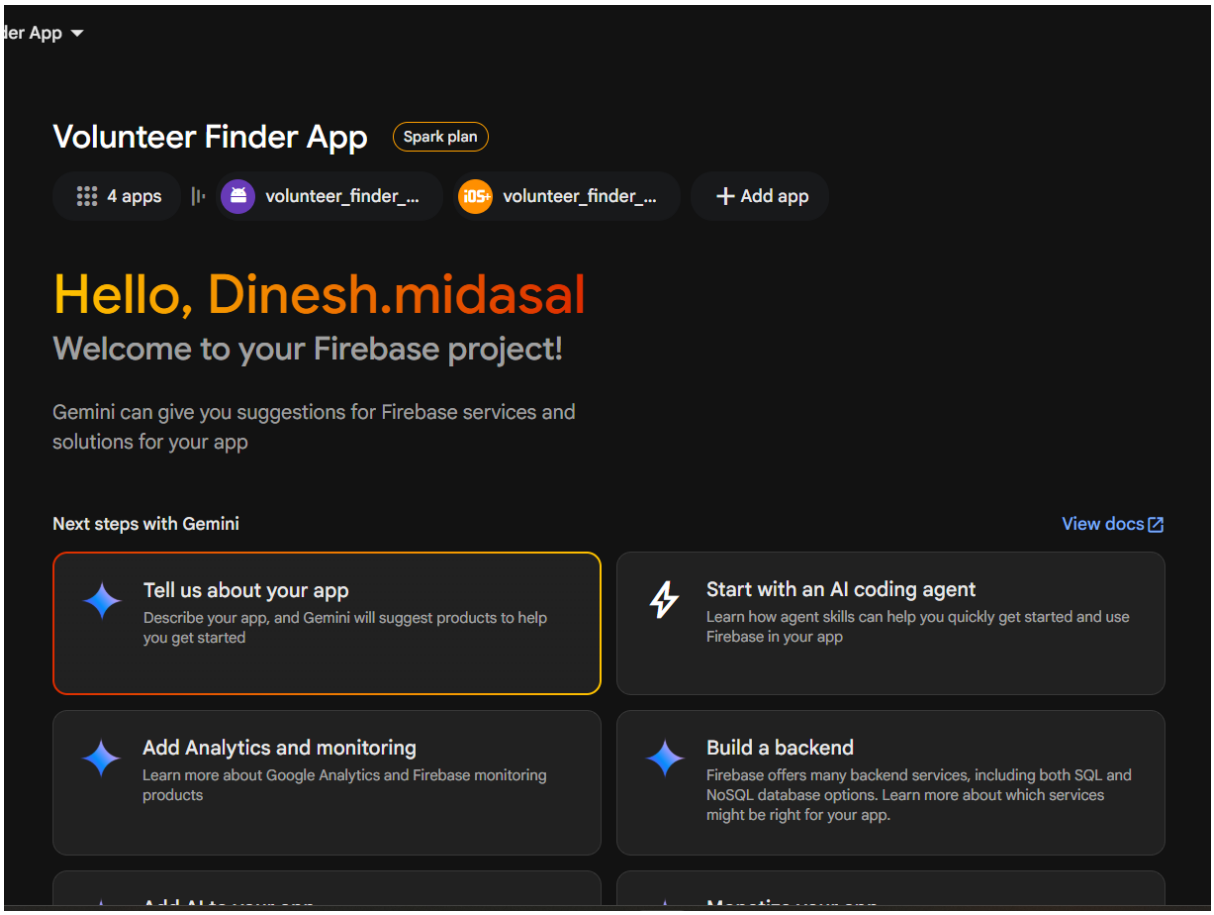
1. **Create a Firebase Project:**
   - Go to the Firebase Console.
   - Click on *Add Project* and follow the on-screen instructions.



- Firebase Console dashboard.

2. **Add Your App to Firebase:**
   - Choose the platform (Android, iOS, or Web).
   - For Android, provide the package name and optionally the app nickname and SHA-1 key.
   - Download the google-services.json (for Android) or GoogleService-Info.plist (for iOS) and add it to your project.

3. **Add Firebase SDK to Your Project:**
   - Open the pubspec.yaml file and add Firebase dependencies:



Adding a new app in Firebase.

```
dependencies:
  firebase_core: ^2.24.0
  firebase_auth: ^5.16.0
  cloud_firestore: ^5.16.0
```

- Run flutter pub get to install the packages.

4. **Initialize Firebase in Your App:**
   - In main.dart, ensure Firebase is initialized before running the app:

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp();
  runApp(MyApp());}
```

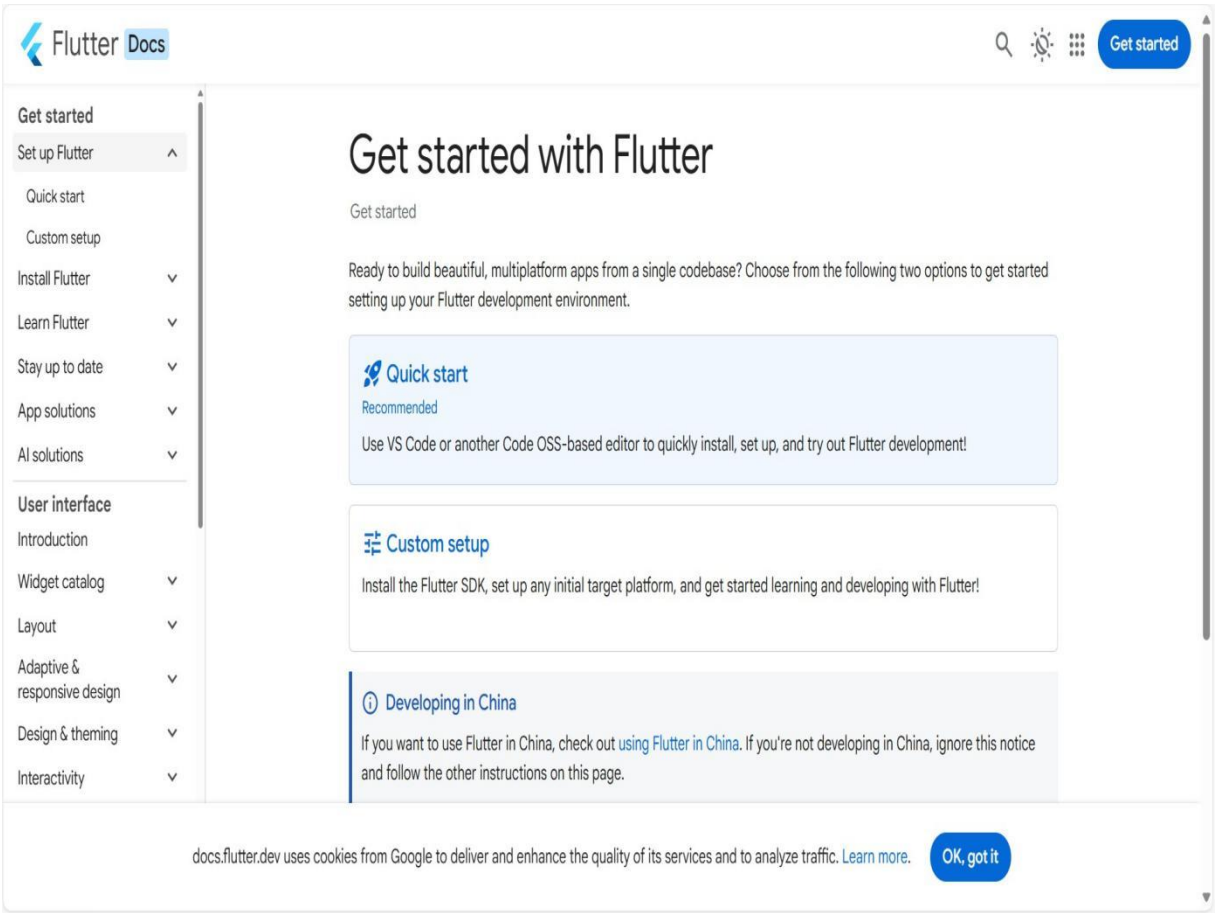
5. **Verify Installation:**
   - Run the app on an emulator or device.
   - Check the console for successful Firebase initialization.

4 Installation of Flutter (with PATH Setup)

Download Flutter SDK

Step 1:

1. Visit: https://docs.flutter.dev/get-started/install.
2. Choose OS (Windows/Mac/Linux).
3. Download Flutter SDK (Stable Channel).
4. Extract it:
   - Windows: C:\src\flutter
   - Linux/Mac: $HOME/flutter



Insatllation of flutter

Add Flutter to PATH

Step 2:

- Windows:
  1. Go to Control Panel → System → Advanced system settings.
  2. Environment Variables → Edit Path.

3. Add: C:\src\flutter\bin.

- Linux/Mac:

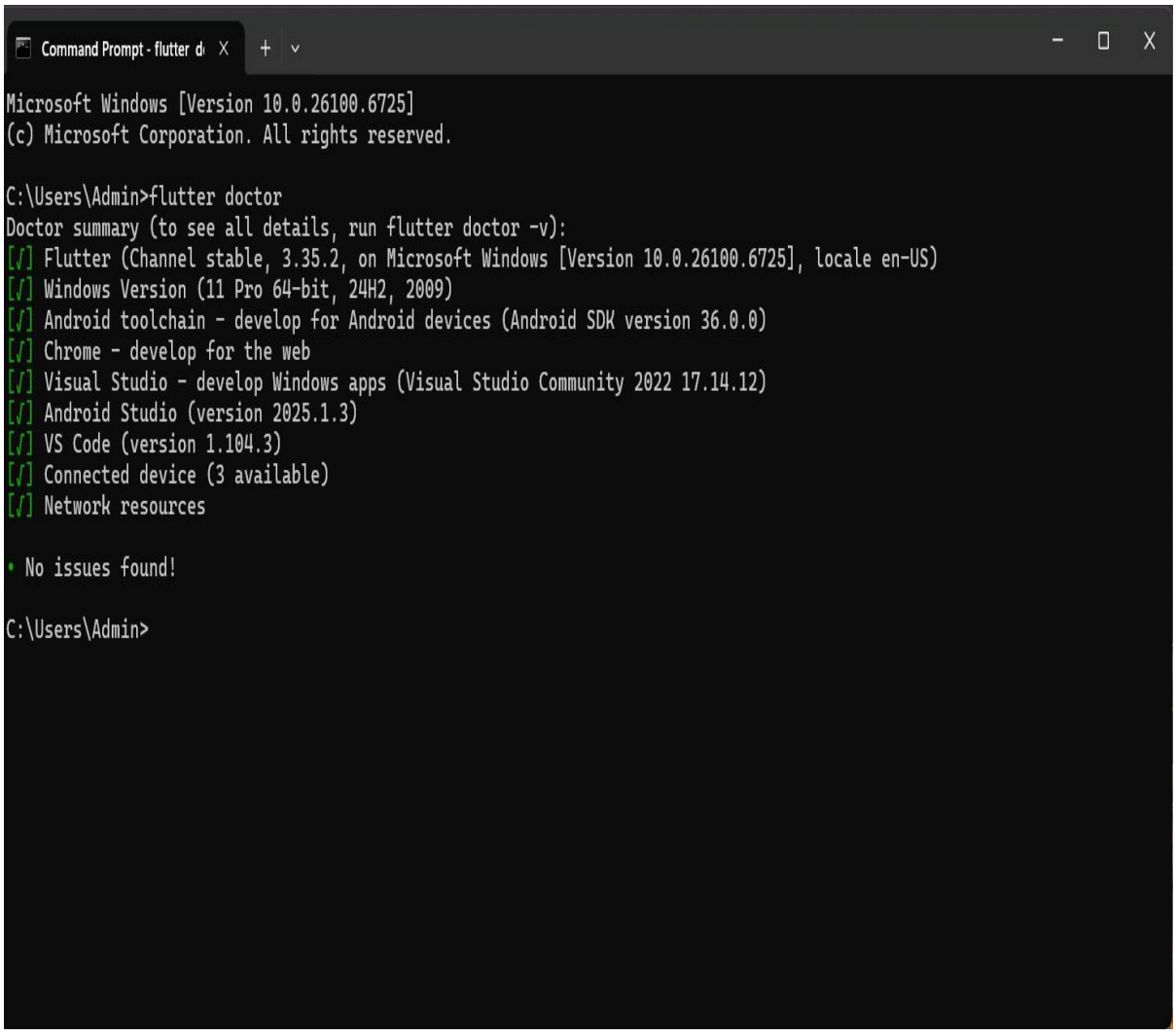
```
export PATH="$PATH:$HOME/flutter/bin"
```

Add this line to .bashrc or .zshrc.

Verify Installation

Step 3:

```
flutter doctor
```



flutter doctor Command Showing Environment Setup

Installation of Android Studio

Step 1: Download and Install

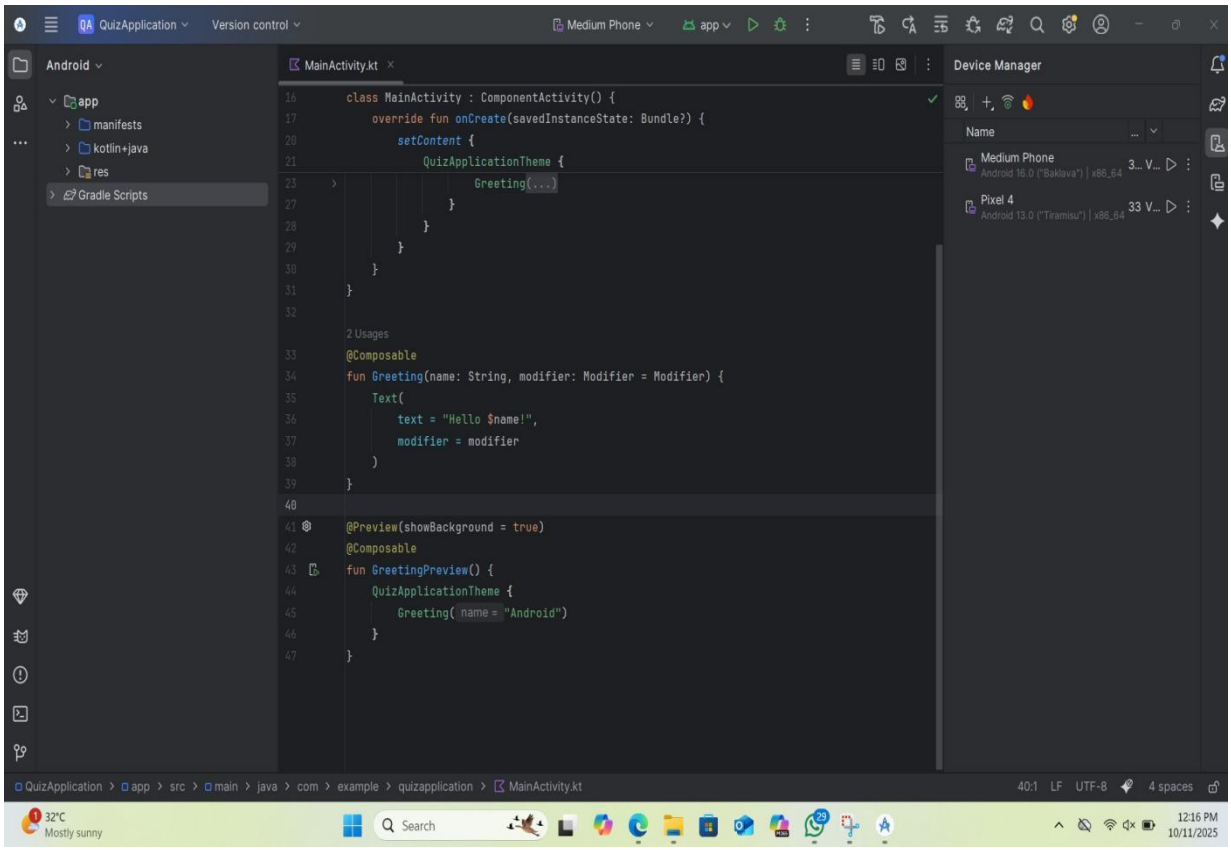
1. Go to: https://developer.android.com/studio.
2. Download Android Studio.
3. Install with default settings.

Step 2: Install Plugins

1. Open Android Studio.
2. Go to File → Settings → Plugins.
3. Install Flutter (Dart installs automatically).

Step 3: Configure SDK Path

1. Open SDK Manager.
2. Verify Android SDK location.
3. Install required SDK platforms & tools.



Android Studio Interface Showing Project Workspace

Emulator Setup and Running

1. Open Android Studio.
2. Go to Tools → AVD Manager.
3. Click Create Virtual Device.
4. Choose a device model (e.g., Pixel 6).
5. Select a system image (Android 13 or latest).
6. Finish and launch emulator.



the emulator image.

5 Installation of Dart SDK in Windows

Dart is a programming language developed by Google, mainly used for building mobile, desktop, web, and server applications. The following steps describe how to install the Dart SDK on a Windows system.

1. **Download the Dart SDK:**
   - Go to the official Dart SDK website: https://dart.dev/get-dart.
   - Under the **Windows** section, choose the appropriate version (64-bit recommended) and download the ZIP file.
2. **Extract the Dart SDK:**
   - After downloading, extract the ZIP file to a preferred location, for example: C:\dart_sdk
3. **Set the Environment Variable:**
   - Right-click on **This PC** → **Properties** → **Advanced system settings**.
   - Click on **Environment Variables**.
   - Under **System variables**, select **Path** → **Edit**.
   - Click **New** and add the path to the bin folder inside Dart SDK, e.g., C:\dart_sdk\bin
4. **Verify the Installation:**
   - Open the **Command Prompt**.
   - Run the following command to verify that Dart is installed successfully:

   ```
   dart --version
   ```

5. **Output:** The terminal should display the installed Dart version, for example:

   ```
   Dart SDK version: 3.2.0 (stable)
   ```

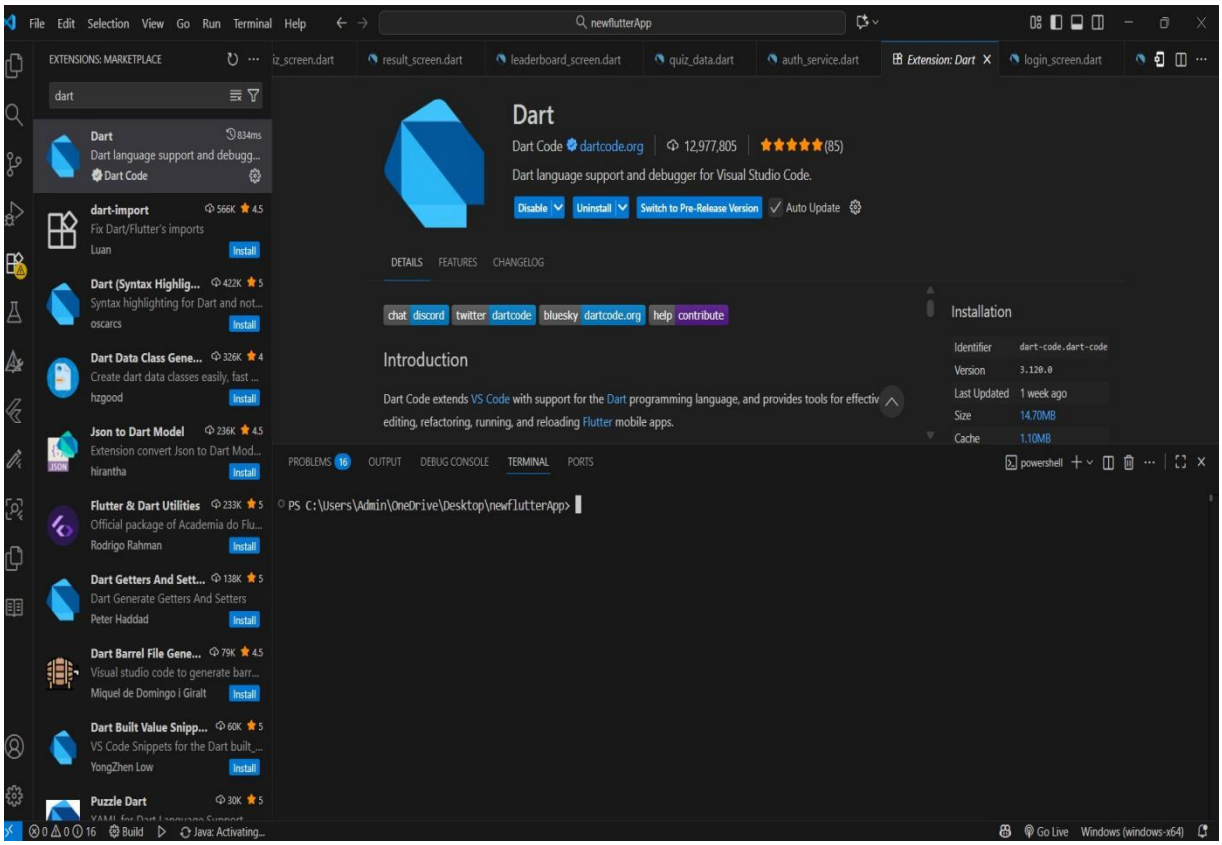
Once the Dart SDK is installed and configured, you can use it to compile and run Dart programs or build Flutter applications.

6 Installing Visual Studio Code

Visual Studio Code (VS Code) is a lightweight yet powerful open-source code editor developed by Microsoft. It is widely used for Flutter and Dart development because of its excellent support for extensions, debugging, IntelliSense, and integration with the Flutter SDK. By installing VS Code, developers can write, test, and debug Flutter applications efficiently across multiple platforms.

Steps to Install VS Code and Set Up for Flutter Development

1. **Download VS Code:** Visit the official Visual Studio Code website at https://code.visualstudio.com/ and download the version suitable for your operating system (Windows, macOS, or Linux).
2. **Install VS Code:** Run the downloaded installer and complete the installation using the default settings. This ensures all necessary components, including PATH configuration, are properly set up.
3. **Install Extensions:** After launching VS Code, navigate to the Extensions panel using Ctrl + Shift + X. Search for and install the following essential extensions that are flutter,dart.
4. **Create a New Flutter Project:** Press Ctrl + Shift + P to open the Command Palette and search for Flutter: New Project. Select a project type (e.g., Application), specify a project name (e.g., quiz_leaderboard), and choose a desired directory to create your new project.
5. **Open the Project in VS Code:** Once the project is created, VS Code automatically opens the folder. You can explore folders such as lib/, android/, and ios/ which are automatically generated by Flutter.



Installing the Dart and Flutter extensions in Visual Studio Code

7 Creating a New Flutter Project

Flutter is a framework developed by Google for building cross-platform applications using the Dart programming language. Before creating a new Flutter project, ensure that the Flutter SDK is installed and added to your system PATH.

Steps to Create a New Flutter Project

1. **Open Command Prompt or Terminal.**
2. **Navigate to the desired folder:**

   ```
   cd C:\Users\<your-username>\Documents
   ```

3. **Create a new Flutter project:**

   ```
   flutter create my_first_app
   ```

4. **Navigate into the project directory:**

   ```
   cd my_first_app
   ```

5. **Open the project in Visual Studio Code (or any preferred IDE):**

   ```
   code .
   ```

6. **Run the Flutter application:**

```
flutter run
```

7. **Verify successful installation:** Once the app runs, a default Flutter counter application appears on the emulator or connected device.

Project Structure

After running the above commands, Flutter automatically generates the following project structure:

- lib/ – Contains the main Dart source code.
- android/, ios/, web/ – Platform-specific code.
- pubspec.yaml – Configuration file for dependencies and assets.
- test/ – Contains test files for unit and widget testing.
- assets/ – Directory to store images, fonts, and other static resources (added manually when needed).

8 Dart Basic Programming

Dart is a modern, object-oriented programming language developed by Google. It is optimized for building client applications, particularly mobile apps using the Flutter framework. Dart provides features like strong typing, asynchronous programming, and a rich standard library, making it both easy to learn and powerful for application development.

The following sections explain the fundamental programming concepts in Dart.

Variables

Variables are containers that hold data values. In Dart, every variable has a **name**, a **type**, and a **value**. Dart is a statically typed language, which means the data type of a variable is known at compile-time. However, Dart also supports type inference using the var keyword.

- **String:** Stores textual data.
- **int:** Stores whole numbers.
- **double:** Stores decimal numbers.
- **bool:** Stores Boolean values (true or false).
- **var:** Automatically infers the data type from the assigned
- **const:** Declares a compile-time constant — the value must be known and fixed at compile time.
- **final:** Declares a variable whose value can be set only once, but is determined at runtime.
- Dart is **type-safe**, meaning you cannot assign a value of a different type once declared.

Code:

```
void main() {
  String name = 'Lalithya';
  int age = 22;
  double height = 5.6;
  bool isStudent = true;
  print('Name: $name');
  print('Age: $age');
  print('Height: $height');
  print('Student: $isStudent');}
```

Output:

```
Name: Lalithya
Age: 22
Height: 5.6
Student: true
```

Conditional Statements

Conditional statements are used to make decisions in a program. They allow different actions to be executed based on whether certain conditions are true or false. In Dart, conditional logic is implemented using **if**, else **if**, and else statements.

- **if:** Executes a block if the condition is true.
- **else if:** Checks another condition if the previous one is false.

Code:

```
void main(){
  int marks = 85;
  if (marks >= 90) {
    print('Grade:A+');
  } else if (marks >= 75){
    print('Grade: A');
  }   else   {
    print('Grade B');
  }
  }
```

Output:

```
Grade: A
```

Loops

Loops are used to repeatedly execute a block of code as long as a specified condition is true. They help automate repetitive tasks and reduce redundancy. Dart supports the following types of loops:

* **for loop:** Repeats code a fixed number of times.
* **while loop:** Repeats while a condition is true.
* **do-while loop:** Executes code once, then checks the condition.

Code:

```
void main() {
  for (int i = 1; i <= 5; i++) {
    print('Iteration: $i');
  }
}
```

Output:

```
Iteration: 1
Iteration: 2
Iteration: 3
Iteration: 4
Iteration: 5
```

Lists

A List in Dart is an ordered collection of elements. Lists are similar to arrays in other programming languages. They can store multiple values of the same or different data types and allow random access using indexes. **Code:**

```
void main() {
  List<String>fruits=['Apple', 'Banana', 'Mango'];
  fruits.add('Orange');
  print('Fruits List: $fruits');}
```

Output:

```
Fruits List: [Apple, Banana, Mango, Orange]
```

Maps

Maps in Dart are used to store data as key-value pairs, where each key is unique. They are similar to dictionaries in Python or hash maps in Java. Maps allow fast data retrieval using keys.

Code:

```
void main() {
  Map<String, int> scores = {
    'Alice': 90,
    'Bob': 80,
    'Charlie ':     95};
  scores.forEach((name, mark) {
    print('$name scored $mark marks');
  });}
```

Output:

```
Alice scored 90 marks
Bob scored 80 marks
Charlie scored 95 marks
```

Functions

Functions are reusable blocks of code designed to perform specific tasks. They help make programs modular and easier to maintain. In Dart, functions can take parameters and return values.

Code:

```
int addNumbers(int a, int b) {
  return a + b;}
void main() {
  int result = addNumbers(5, 7);
  print('Sum: $result');}
```

Output:

```
Sum: 12
```

9 Case Study on Volunteer Finder App

The **Volunteer Finder App** is an interactive mobile application built using Flutter and Dart. It allows users to find suitable volunteers,view volunteer details request help, and connect with people who are willing to provide support. This project demonstrates a practical application of Flutter's **UI design principles, responsive layouts, navigation, form handling, state management, and Firebase integration**.In many communities, people who need assistance often find it difficult to identify suitable volunteers quickly, while volunteers may not have an easy platform to offer their services. The aim of this project is to create a **cross-platform mobile application** that provides users with:

Key Features

- **User Authentication:** Secure login and registration functionality for users.
- **Volunteer Finder Interface:** Displays volunteer details with available services and provides instant access to request assistance.
- **Leaderboard:** Shows real-time user scores fetched from a backend API.
- **Profile Management:** Allows users to edit and update their personal details.
- **Responsive UI:** Adapts seamlessly to different screen sizes and orientations.
- **Animations:** Smooth transitions between screens for better user experience.
- **Custom Widgets:** Reusable UI components like buttons and question cards.
- **State Management:** Efficient handling of dynamic quiz and leaderboard data.

Technologies Used

- **Flutter SDK:** For building cross-platform UI and managing widgets.
- **Dart Language:** The core programming language for Flutter development.
- **VS Code:** Integrated development environment used for coding and debugging.
- **Android Emulator:** Used for testing and visualizing the app during development.
- **REST API:** For fetching and updating leaderboard data dynamically.
- **Provider Package:** For state management and real-time UI updates.
- **Firebase / Mock Backend:** For user authentication and data storage (if used).
- **GitHub:** For version control and project collaboration.

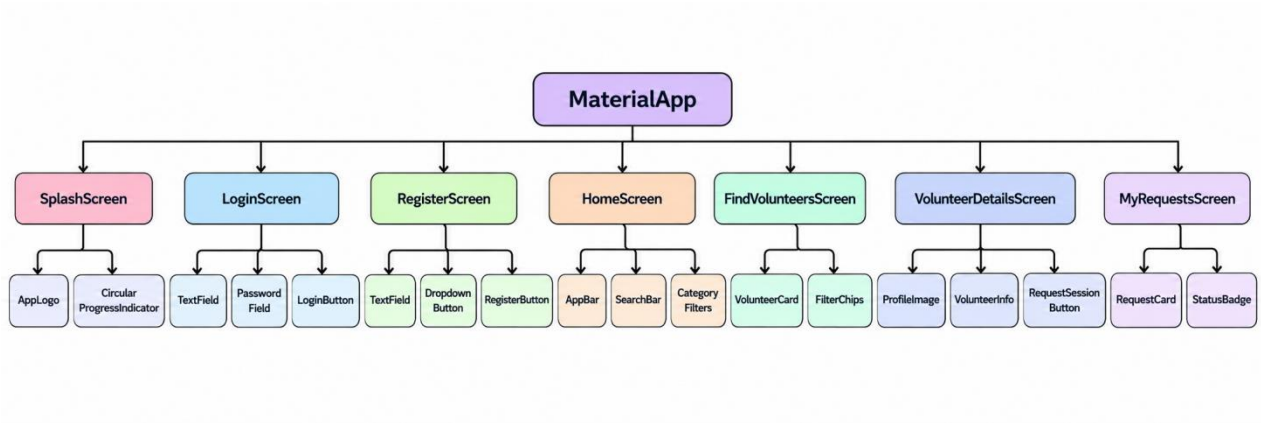
10 Exploring Flutter Widgets

Objective: To explore basic widgets like Text, Image, and Container.

Relation to Project: Used throughout the Volunteer Finder App to create user interface elements such as volunteer profile cards, service category tiles, availability indicators, and session request details.

Description: The VolunteerCard widget is a custom Flutter component designed to display volunteer information along with the services they offer. It presents the volunteer's name, service category, description, and availability status in a structured card layout. Users can view volunteer details and identify suitable volunteers for their needs. The widget uses basic Flutter widgets such as Text, Image, and Container to create a clear and user-friendly interface.

Widgets Flow:



Simplified Widget Tree of the Volunteer Finder App

Widget Structure: The widget uses a `Card` with rounded corners and padding to display volunteer information. Inside the card:

- The volunteer's name is displayed in bold text.
- The volunteer's service category, description, location, and availability are displayed using styled text.
- The volunteer's profile picture is displayed using an `Image` widget.
- A Request Session button is provided with a colored background, white text, and rounded corners to allow users to send session requests.

question card code:

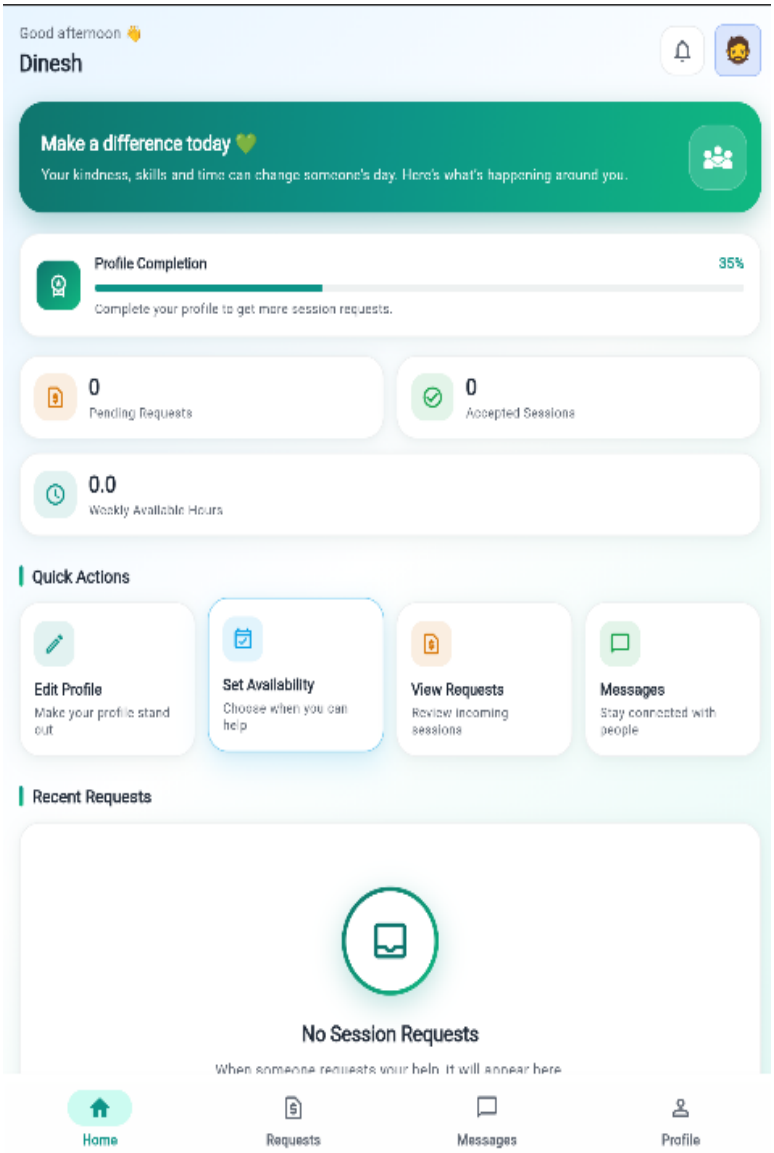
```
import 'package:flutter/material.dart';

class VolunteerCard extends StatelessWidget {
  final String name;
  final String category;

  const VolunteerCard({
    super.key,
    required this.name,
    required this.category,
  });
  @override
  Widget build(BuildContext context) {
```

```
        return Card(
          child: ListTile(
            leading: const Icon(Icons.person),
            title: Text(
              name,
              style: const TextStyle(fontWeight: FontWeight.bold),
            ),
            subtitle: Text(category),
            trailing: ElevatedButton(
              onPressed: () {},
              child: const Text('Request'),
            ),
          ),
        );
      }
    }
```

Output:



Volunteer Screen

Description:

The VolunteerCard widget is a custom Flutter component used to display an individual volunteer's details in the Volunteer Finder App. It shows the volunteer's name, service category, and availability in a visually structured card format..

Functionality:

Each volunteer entry is represented using a Card and ListTile widget. The volunteer's profile icon is displayed on the left, the volunteer's name appears as the title, and the service category is shown below the name. A Request button is provided on the right side of the tile to allow users to initiate a session request.

Constructor Parameters:

- name: The name of the volunteer.
- category: The service category offered by the volunteer.

Widget Structure:

The widget uses the `ListTile` layout with the following components:

- **leading:** An `Icon` widget displaying a person icon.
- **title:** A bold `Text` widget displaying the volunteer's name.
- **subtitle:** A `Text` widget displaying the volunteer's service category.
- **trailing:** An `ElevatedButton` widget labeled Request to initiate a volunteer session request.

LeaderboardTile code:

```
import 'package:flutter/material.dart';
import '../services/volunteer_service.dart';
import 'dashboard_screen.dart';
class FindVolunteersScreen extends StatefulWidget {
  const FindVolunteersScreen({super.key});
  @override
  State<FindVolunteersScreen> createState() =>
      _FindVolunteersScreenState();
}
class _FindVolunteersScreenState
    extends State<FindVolunteersScreen> {
  late Future<List<Map<String, dynamic>>> _volunteersFuture;
  String searchQuery = "";
  String categoryFilter = "All";
  @override
  void initState() {
    super.initState();
    _volunteersFuture = VolunteerService.getAll();
  }
  IconData _availabilityIcon(bool available) {
    return available ? Icons.check_circle : Icons.schedule;
  }
  Color _availabilityColor(bool available) {
    return available ? Colors.green : Colors.orange;
  }
```

```
Widget _buildAnimatedEntry({
   required int index,
   required Widget child,
 }) {
   return TweenAnimationBuilder<double>(
     tween: Tween(begin: 0, end: 1),
     duration: Duration(milliseconds: 400 + (index * 100)),
     curve: Curves.easeOut,
     builder: (context, value, _) {
       return Opacity(
         opacity: value,
         child: Transform.translate(
           offset: Offset(0, 20 * (1 - value)),
           child: child,
         ),);},);}
 List<Map<String, dynamic>> _filterVolunteers(
   List<Map<String, dynamic>> volunteers,
 ) {
   return volunteers.where((volunteer) {
     final name =
         (volunteer['name'] ?? '').toString().toLowerCase();
     final category =
         (volunteer['category'] ?? '').toString();
     final matchesSearch =
         name.contains(searchQuery.toLowerCase());
     final matchesCategory = categoryFilter == 'All' ||
         category == categoryFilter;
     return matchesSearch && matchesCategory;
   }).toList();
 }
 @override
 Widget build(BuildContext context) {
   return Scaffold(
     appBar: AppBar(
       title: const Text('Find Volunteers'),
       actions: [
         IconButton(
           icon: const Icon(Icons.home),
           tooltip: 'Go to Dashboard',
           onPressed: () {
             Navigator.pushReplacement(
               context,
               MaterialPageRoute(
                 builder: (_) => const DashboardScreen(),
               ),);},),],),
     body: Container(
       decoration: const BoxDecoration(
         gradient: LinearGradient(
           colors: [
             Colors.purpleAccent,
             Colors.deepPurple,
             Colors.pinkAccent,
```

```
      ],
      begin: Alignment.topLeft,
      end: Alignment.bottomRight,
    ),
  ),
  child: Column(
    children: [
      Padding(
        padding: const EdgeInsets.all(12),
        child: TextField(
          onChanged: (value) {
            setState(() => searchQuery = value);
          },
          decoration: InputDecoration(
            hintText: 'Search volunteers...',
            prefixIcon: const Icon(Icons.search),
            filled: true,
            fillColor: Colors.white,
            border: OutlineInputBorder(
              borderRadius: BorderRadius.circular(12),
            ),
          ),
        ),
      ),
      Padding(
        padding: const EdgeInsets.symmetric(
          horizontal: 12,
        ),
        child: DropdownButtonFormField<String>(
          value: categoryFilter,
          decoration: const InputDecoration(
            filled: true,
            fillColor: Colors.white,
            labelText: 'Service Category',
            border: OutlineInputBorder(),
          ),
          items: [
            'All',
            'Education',
            'Health',
            'Environment',
          ].map((category) {
            return DropdownMenuItem(
              value: category,
              child: Text(category),
            );}).toList(),
          onChanged: (value) {
            setState(() {
              categoryFilter = value ?? 'All';
            });}, ),),
```

```
Expanded(
  child: FutureBuilder<
      List<Map<String, dynamic>>>(
    future: _volunteersFuture,
    builder: (context, snapshot) {
      if (snapshot.connectionState ==
          ConnectionState.waiting) {
        return const Center(
          child: CircularProgressIndicator(
            color: Colors.white,
          ),
        );
      }

      if (snapshot.hasError) {
        return const Center(
          child: Text(
            'Unable to load volunteers.',
            style: TextStyle(color: Colors.white),
          ),
        );
      }

      final volunteers =
          _filterVolunteers(snapshot.data ?? []);

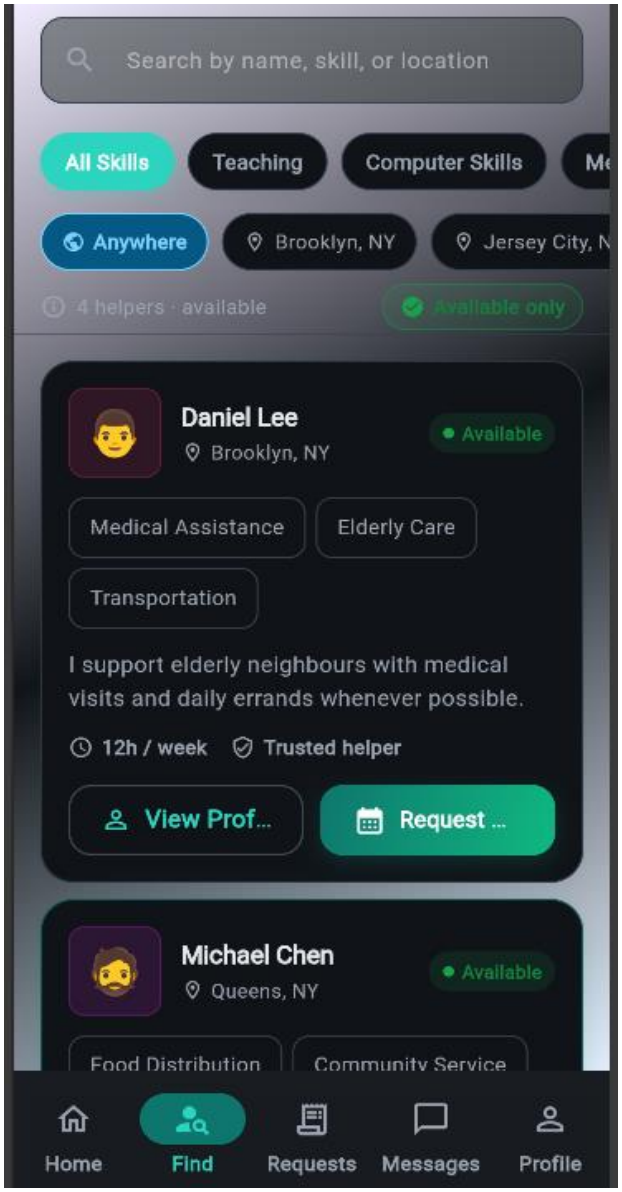
      if (volunteers.isEmpty) {
        return const Center(
          child: Text(
            'No volunteers found.',
            style: TextStyle(color: Colors.white),
          ),
        );
      }

      return ListView.builder(
        itemCount: volunteers.length,
        itemBuilder: (context, index) {
          final volunteer = volunteers[index];
          final available =
              volunteer['available'] == true;

          return _buildAnimatedEntry(
            index: index,
            child: Card(
              margin: const EdgeInsets.symmetric(
                horizontal: 12,
                vertical: 6,
              ),
              child: ListTile(
                leading: const CircleAvatar(
                  child: Icon(Icons.person),
                ),
```

```
                  title: Text(
                    volunteer['name']?.toString() ??
     'Volunteer',
     style: const TextStyle(
    fontWeight: FontWeight.bold,
    ),
  ),
      subtitle: Text(
       '${volunteer['category'] ?? 'General'}\n'
        '${volunteer['description'] ?? ''}',
                  ),
      isThreeLine: true,
       trailing: Icon(
      _availabilityIcon(available),
       color: _availabilityColor(available),
);},),),],),),);}}
```

Output:



Requester Screen

11 Responsive UI Design

Objective: Design a responsive layout using Row, Column, and MediaQuery.
Relation to Project: The dashboard adapts to screen sizes across devices.
Description :

The DashboardScreen in the Volunteer Finder App implements responsive UI design principles in Flutter, ensuring that the layout adapts to different screen sizes and device orientations. Widgets such as SafeArea and SingleChildScrollView help prevent content overflow and keep important information visible on devices with notches or smaller displays.The interface uses flexible layout widgets such as Row, Column, Expanded, and Container to organize volunteer details, service categories, and session request options. The MediaQuery widget can be used to adjust spacing, padding, and card layouts according to the available screen width and height.The volunteer profile cards and service category sections can reposition themselves based on the available screen space. Scrolling allows users to access all volunteer details and session request options on smaller screens.Animations and smooth transitions can further enhance user engagement while maintaining a responsive interface. Overall, the Volunteer Finder App aims to provide a visually consistent, user-friendly, and adaptable experience across different devices.

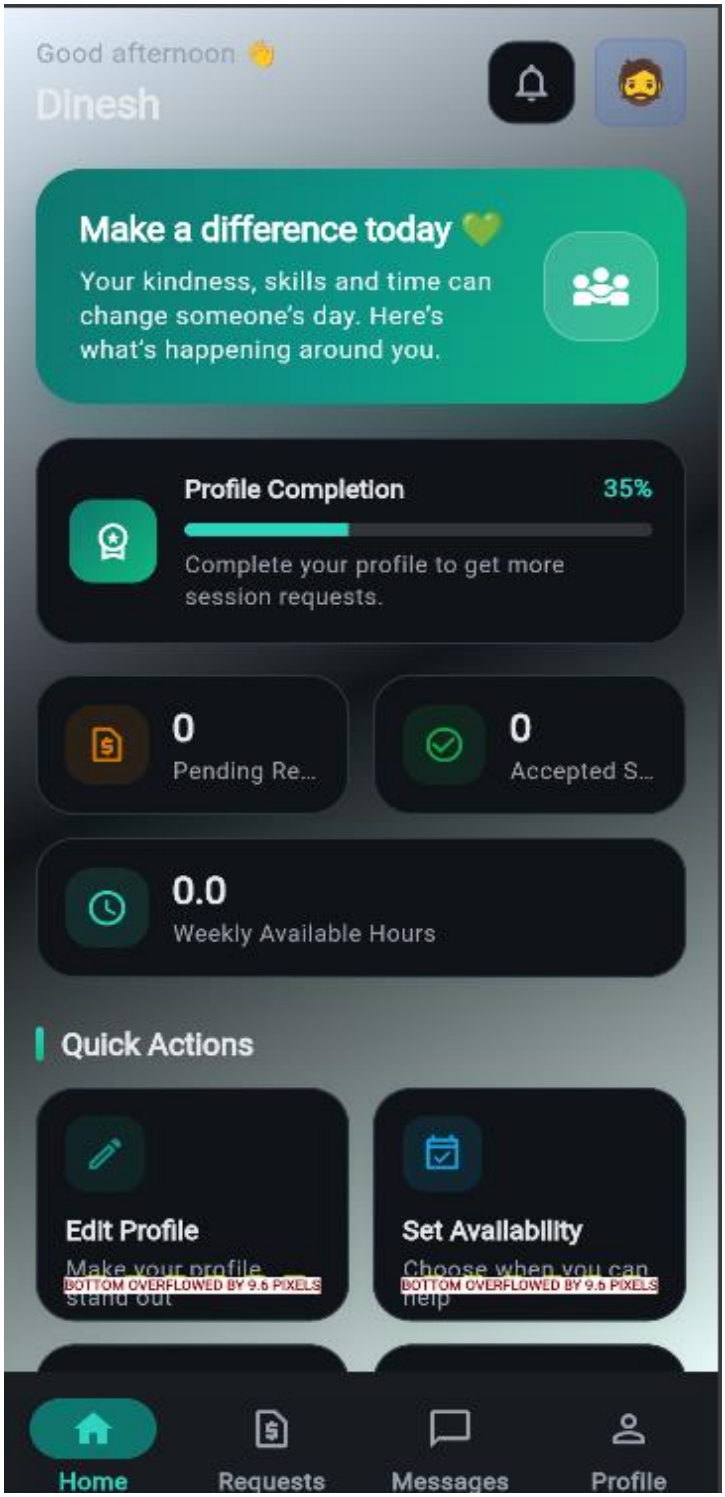
Dashboard Screen code:

```
import 'package:flutter/material.dart';
class DashboardScreen extends StatefulWidget {
  const DashboardScreen({super.key});
  @override
  State<DashboardScreen> createState() =>
      _DashboardScreenState();
}
class _DashboardScreenState extends State<DashboardScreen> {
  @override
  Widget build(BuildContext context) {
    final width = MediaQuery.of(context).size.width;
    return Scaffold(
      appBar: AppBar(
        title: const Text('Volunteer Finder'),
        backgroundColor: Colors.deepPurple,
        foregroundColor: Colors.white,
      ),
      body: SafeArea(
        child: SingleChildScrollView(
          padding: const EdgeInsets.all(16),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              const CircleAvatar(
                radius: 40,
                backgroundColor: Colors.deepPurple,
                child: Icon(
                  Icons.volunteer_activism,
                  size: 45,
                  color: Colors.white,
                ),
              ),
              const SizedBox(height: 16),
              const Text(
                'Welcome to Volunteer Finder!',
                style: TextStyle(
                  fontSize: 22,
```

```
                    fontWeight: FontWeight.bold,
                  ),
                ),
                const SizedBox(height: 20),
                Wrap(
                  spacing: 12,
                  runSpacing: 12,
                  children: [
                    _buildCard(
                      width,
                      Icons.search,
                      'Find Volunteers',
                      Colors.deepPurple,
                    ),
                    _buildCard(
                      width,
                      Icons.assignment,
                      'My Requests',
                      Colors.teal,
                    ),
                    _buildCard(
                      width,
                      Icons.person,
                      'My Profile',
                      Colors.orange,
                    ),
                  ],
                ),
              ],
            ),
          ),
        ),
      );
    }

    Widget _buildCard(
      double screenWidth,
      IconData icon,
      String title,
      Color color,
    ) {
      return SizedBox(
        width: screenWidth > 600
            ? (screenWidth - 56) / 2
            : screenWidth - 32,
        child: Card(
          elevation: 4,
          child: ListTile(
            leading: Icon(icon, color: color, size: 32),
            title: Text(
              title,
              style: const TextStyle(
                fontWeight: FontWeight.bold,
              ),
            ),
            trailing: const Icon(Icons.arrow_forward_ios),
          ),
        ),
      );
    }
  }
```

Output:



Dashboard Screen

12 Navigation Between Screens

Objective: To implement navigation using Navigator and named routes.
Relation to Project: Used to move between the Login, Register, Dashboard, Find Volunteers, Volunteer Details, and My Requests screens.

Explanation:

- Users start at the LoginScreen.
- After successful authentication, users are navigated to the DashboardScreen.
- From the DashboardScreen, users can:
- Access FindVolunteersScreen to search for volunteers.
- Access VolunteerDetailsScreen to view volunteer information.
- Access MyRequestsScreen to track session requests.
- Access RegisterScreen to create a new account
- Logging out from the dashboard:
- Redirects the user back to the LoginScreen.
- Clears the navigation stack to prevent returning to authenticated screens using the back button.

Sample Code:

```
import 'package:flutter/material.dart';
void main() {
  runApp(MaterialApp(
    initialRoute: '/',
    routes: {
      '/': (context) => const LoginScreen(),
      '/dashboard': (context) => const DashboardScreen(),
      '/findVolunteers': (context) =>
          const FindVolunteersScreen(),
      '/myRequests': (context) => const MyRequestsScreen(),
    },
  ));
}
Navigator.pushNamed(context, '/findVolunteers');
Navigator.pushNamed(context, '/myRequests');
Navigator.pop(context);
Navigator.pushNamedAndRemoveUntil(
  context,
  '/',
  (route) => false,
);
```

13 Stateful and Stateless Widgets

Objective: Understand widget lifecycles and state management.

Relation to Project: Used in the Volunteer Finder App to display volunteer details using stateless widgets and manage dynamic data using stateful widgets.

Description: In Flutter, widgets are the building blocks of the user interface and are classified as either Stateless or Stateful, depending on whether they maintain internal state that can change during the widget's lifecycle

.* **Stateless Widgets:**

Stateless widgets do not maintain mutable internal state and are immutable once created. They rebuild when their parent widget provides updated configuration.

In the Volunteer Finder App, examples include:

- **VolunteerCard** – Displays a volunteer's name and service category using input parameters.
- **CustomButton** – Displays a button with a predefined label and style, with actions
- handled through callbacks.

Stateless widgets are used when the UI content depends only on the input parameters and does not change dynamically within the widget itself.

Syntax for StatelessWidget:

```
class VolunteerCard extends StatelessWidget {
              final String name;
            final String category;

             const VolunteerCard({
                   super.key,
                required this.name,
             required this.category,
                      });

                    @override
          Widget build(BuildContext context) {
                 return Container(
                    child: Column(
                       children: [
                        Text(name),
                       Text(category),
                             ],
                           ),
                         );
                        }
                       }
```

* **Stateful Widgets:**

- Stateful widgets maintain mutable state that can change over time. They can rebuild their interface whenever their internal state changes. In the Volunteer Finder App, examples include:
- **DashboardScreen** –Displays the logged-in user's information and updates when profile details change.
- **FindVolunteersScreen** – Tracks search text and selected service category to filter volunteers dynamically

Stateful widgets are essential for interactive UI components that respond to user input or asynchronous data changes.

Syntax for Stateful Widget:

```
class FindVolunteersScreen extends StatefulWidget {
  const FindVolunteersScreen({super.key});

  @override
  State<FindVolunteersScreen> createState() =>
      _FindVolunteersScreenState();
}

class _FindVolunteersScreenState
    extends State<FindVolunteersScreen> {
  @override
  Widget build(BuildContext context) {
    return Container(
      // Volunteer Finder UI code here
    );
  }
}
```

14 Custom Widgets and Styling

Objective: Create custom reusable widgets.

Description:

The Volunteer Finder App uses custom widgets and consistent styling to improve modularity, maintainability, and visual appearance. Custom widgets encapsulate UI components that can be reused across multiple screens, reducing code duplication and maintaining a uniform design

* **Custom Widgets:**

- VolunteerCard: A reusable card widget that displays a quiz ques-tion along with multiple options. Each option is a styled button that triggers a callback when selected.
- RequestTile: A reusable list tile to display user rank, name, and score with a circular avatar and trailing score.
- CustomButton: A reusable button component with configurable text, icon, color, and onPressed action. Used in registration, login, and navigation buttons.

* **Styling Approach:**

- Consistent color scheme using shades of deepPurple and complemen-tary accent colors for gradients (e.g., option cards and dashboard back-grounds).
- Rounded corners on cards, buttons, and text fields for a modern look.
- Gradients and shadows applied to navigation cards on the dashboard for depth and visual hierarchy.
- Bold and large text styles for headings (username, quiz question, titles) to improve readability.
- Padding, margins, and spacing are used consistently to maintain visual balance and responsive layout across different screen sizes.

- Icons combined with text in buttons and list tiles enhance usability and quick recognition of actions.

* **Benefits:**

- Enhances code reusability by creating modular components.
- Maintains a consistent look and feel throughout the application.
- Simplifies maintenance and scalability when adding new screens or UI elements.
- Improves readability, accessibility, and user engagement through well-thought-out styling.

Relation to Project: Used for buttons and leaderboard tiles.

Custom Button code:

```
import 'package:flutter/material.dart';
class CustomButton extends StatelessWidget {
  final String text;
  final VoidCallback onPressed;
  final Color color;
  final IconData? icon;
  const CustomButton({
    super.key,
    required this.text,
    required this.onPressed,
    this.color = Colors.deepPurple,
    this.icon,
  });
  @override
  Widget build(BuildContext context) {
    return ElevatedButton.icon(
      style: ElevatedButton.styleFrom(
        backgroundColor: color,
        padding: const EdgeInsets.symmetric(
          horizontal: 20,
          vertical: 14,
        ),
      ),
      icon: icon != null
          ? Icon(icon, size: 20, color: Colors.white)
          : const SizedBox.shrink(),
      label: Text(
        text,
        style: const TextStyle(
          fontSize: 16,
          fontWeight: FontWeight.bold,
          color: Colors.white,
        ),
      ),
      onPressed: onPressed,
    );
  }
```

output:



App Icon

15 Forms and Validation

Objective: Create a form with validation.

Relation to Project: Used in login and registration screens.

Description:

The RegisterScreen implements a user registration form using multiple TextField widgets to collect input for username, email, phone number, password, and confirm password. It demonstrates basic form validation and user feedback mechanisms:

* **Input Validation:**
  - Checks that all fields are filled before submission.
  - Ensures that the password and confirm password fields match.
  - Displays error messages dynamically if validation fails.
* **User Feedback:**
  - Shows a CircularProgressIndicator while registration is in progress.
  - Displays success messages via SnackBar upon successful registration.
* **Navigation:**
  - On successful registration, the user is navigated to the DashboardScreen using Navigator.pushAndRemoveUntil() to clear the navigation stack.
  - Provides a “Back to Login” button to navigate back to the LoginScreen.
* **Code Structure:**
  - Uses TextEditingController for each input field to read and manage user input.
  - The _register() method encapsulates validation logic, loading state management, and calls to AuthService.register() for actual registration.
  - Error messages and loading indicators are updated dynamically using setState(),demonstrating a typical use **stateful widgets** in form.

Registration Screen code:

```
import 'package:flutter/material.dart';
import '../services/auth_service.dart';
import '../widgets/custom_button.dart';
import 'dashboard_screen.dart';
import 'login_screen.dart';

class RegisterScreen extends StatefulWidget {
  const RegisterScreen({super.key});

  @override
  State<RegisterScreen> createState() => _RegisterScreenState();
}

class _RegisterScreenState extends State<RegisterScreen> {
  final _userCtrl = TextEditingController();
  final _emailCtrl = TextEditingController();
  final _phoneCtrl = TextEditingController();
  final _passCtrl = TextEditingController();
  final _confirmPassCtrl = TextEditingController();
  bool _loading = false;
  String? _error;
  @override
  void dispose() {
    _userCtrl.dispose();
    _emailCtrl.dispose();
    _phoneCtrl.dispose();
    _passCtrl.dispose();
    _confirmPassCtrl.dispose();
    super.dispose();
  }
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Volunteer Finder Registration'),
        backgroundColor: Colors.deepPurple,
        foregroundColor: Colors.white,
      ),
      body: SafeArea(
        child: SingleChildScrollView(
          padding: const EdgeInsets.all(24),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              const Icon(
                Icons.volunteer_activism,
                size: 70,
                color: Colors.deepPurple,
              ),
              const SizedBox(height: 16),
              const Text(
                'Create Your Account',
                textAlign: TextAlign.center,
```

```
  style: TextStyle(
    fontSize: 24,
    fontWeight: FontWeight.bold,
  ),
),
const SizedBox(height: 24),
TextField(
  controller: _userCtrl,
  decoration: InputDecoration(
    labelText: 'Username',
    prefixIcon: const Icon(Icons.person),
    border: OutlineInputBorder(
      borderRadius: BorderRadius.circular(12),
    ),
  ),
),
const SizedBox(height: 16),
TextField(
  controller: _emailCtrl,
  keyboardType: TextInputType.emailAddress,
  decoration: InputDecoration(
    labelText: 'Email',
    prefixIcon: const Icon(Icons.email),
    border: OutlineInputBorder(
      borderRadius: BorderRadius.circular(12),
    ),
  ),
),
const SizedBox(height: 16),
TextField(
  controller: _phoneCtrl,
  keyboardType: TextInputType.phone,
  decoration: InputDecoration(
    labelText: 'Phone Number',
    prefixIcon: const Icon(Icons.phone),
    border: OutlineInputBorder(
      borderRadius: BorderRadius.circular(12),
    ),
  ),
),
const SizedBox(height: 16),
TextField(
  controller: _passCtrl,
  obscureText: true,
  decoration: InputDecoration(
    labelText: 'Password',
    prefixIcon: const Icon(Icons.lock),
    border: OutlineInputBorder(
      borderRadius: BorderRadius.circular(12),
    ),
  ),
),
const SizedBox(height: 16),

TextField(
```

```
                  controller: _confirmPassCtrl,
                  obscureText: true,
                  decoration: InputDecoration(
                    labelText: 'Confirm Password',
                    prefixIcon: const Icon(Icons.lock_outline),
                    border: OutlineInputBorder(
                      borderRadius: BorderRadius.circular(12),
                    ),
                  ),
                ),
                const SizedBox(height: 16),

                if (_error != null)
                  Text(
                    _error!,
                    style: const TextStyle(color: Colors.red),
                  ),

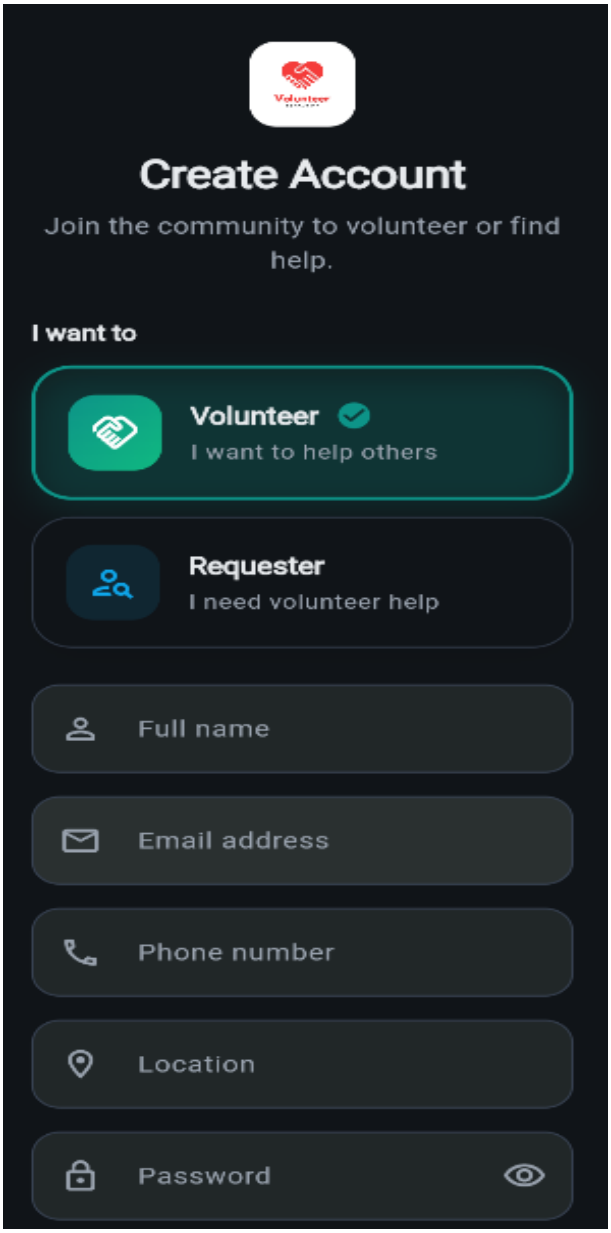
                CustomButton(
                  text: _loading ? 'Registering...' : 'Register',
                  icon: Icons.person_add,
                  onPressed: () {
                    setState(() {
                      _error = _passCtrl.text != _confirmPassCtrl.text
                          ? 'Passwords do not match'
                          : null;
                    });

                    if (_error != null) return;

                    // Connect AuthService registration logic here.
                  },
                ),
                const SizedBox(height: 12),

                TextButton(
                  onPressed: () {
                    Navigator.pushReplacement(
                      context,
                      MaterialPageRoute(
                        builder: (_) => const LoginScreen(),
                      ),
                    );
                  },
                  child: const Text(
                    'Already have an account? Login',
                  ),
                ),
              ],
            ),
          ),
        ),
      );
```

Screenshot:



Register Screen

Description:

The LoginScreen implements user authentication form using TextField widgets for username,password input.

It demonstrates form validation,error handling,and navigation after successful login:

* **Input Validation:**
  - Checks that both username, password fields are not empty before submission.
  - Displays an error message dynamically if validation fails or authentication fails.
* **User Feedback:**
  - Shows a CircularProgressIndicator while the login request is being processed.
  - Displays error messages below the input fields when login fails.
* **Navigation:**
  - On successful login, the user is navigated to the DashboardScreen using Navigator.pushAndRemoveUntil() to clear the navigation stack and prevent back navigation to the login screen.
  - Provides a “Create an Account” button to navigate to the RegisterScreen for new user registration.
* **Code Structure:**
  - Uses TextEditingController to manage the username and password input.
  - The _login() method handles input validation, loading state management, and calls AuthService.login() for authentication.
  - Error messages and loading indicators are updated using setState(), demonstrating a typical use of a **stateful widget** for form handling.

Login Screen code:

```
import 'package:flutter/material.dart';
import '../services/auth_service.dart';
import '../widgets/custom_button.dart';
import 'register_screen.dart';
import 'dashboard_screen.dart';
class LoginScreen extends StatefulWidget {
  const LoginScreen({super.key});
  @override
  State<LoginScreen> createState() => _LoginScreenState();
}
class _LoginScreenState extends State<LoginScreen> {
  final _userCtrl = TextEditingController();
  final _passCtrl = TextEditingController();
  bool _loading = false;
  String? _error;
  @override
  void dispose() {
    _userCtrl.dispose();
    _passCtrl.dispose();
    super.dispose();
  }
  void _login() async {
    if (_userCtrl.text.trim().isEmpty ||
        _passCtrl.text.isEmpty) {
      setState(() {
        _error = 'Please enter username and password';
      });
      return;
    }
    setState(() {
      _loading = true;
      _error = null;
    });
    try {
      // Connect Firebase authentication through AuthService here.
      // This sample navigates to the dashboard after input validation.
      if (!mounted) return;
      Navigator.pushAndRemoveUntil(
        context,
        MaterialPageRoute(
          builder: (_) => const DashboardScreen(),
        ),
        (route) => false,
      );
    } catch (e) {
      setState(() {
        _error = 'Login failed. Please try again.';
      });
    } finally {
      if (mounted) {
        setState(() {
          _loading = false;
        });
      }
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
```

```
        title: const Text('Volunteer Finder Login'),
        backgroundColor: Colors.deepPurple,
        foregroundColor: Colors.white,
      ),
      body: SafeArea(
        child: SingleChildScrollView(
          padding: const EdgeInsets.all(24),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              const Icon(
                Icons.volunteer_activism,
                size: 75,
                color: Colors.deepPurple,
              ),
              const SizedBox(height: 20),
              const Text(
                'Welcome to Volunteer Finder',
                textAlign: TextAlign.center,
                style: TextStyle(
                  fontSize: 23,
                  fontWeight: FontWeight.bold,
                ),
              ),
              const SizedBox(height: 25),

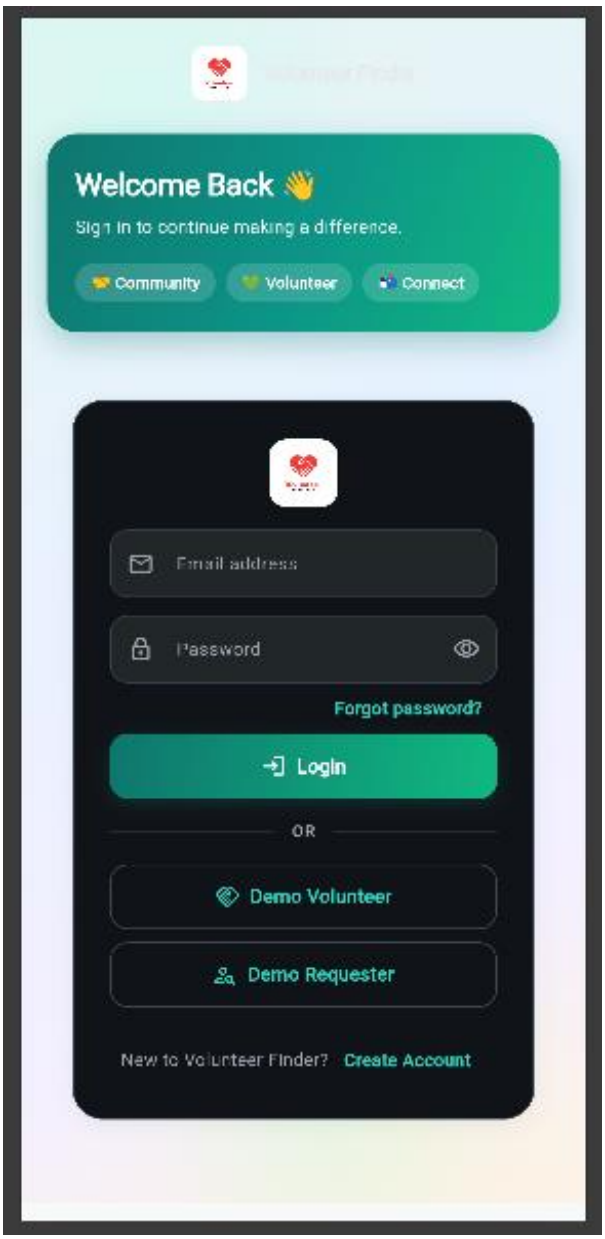

              const SizedBox(height: 16),

              if (_error != null)
                Padding(
                  padding: const EdgeInsets.only(bottom: 12),
                  child: Text(
                    _error!,
                    style: const TextStyle(color: Colors.red),
                  ),
                ),

                            const SizedBox(height: 12),

              CustomButton(
                text: 'Create an Account',
                icon: Icons.person_add,
                color: Colors.grey,
                onPressed: () {
                  Navigator.push(
                    context,
                    MaterialPageRoute(
                      builder: (_) => const RegisterScreen(),
                    ),
                  );
                },
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

Output:



Login Screen

16 Animations

Objective: Add basic UI animations.

Relation to Project: Used for smooth transitions between screens.

Description:

Your Flutter project incorporates animations primarily in the DashboardScreen to enhance user interaction and provide a more engaging experience. The animations include:

* **Avatar Bounce Animation:**
  - Implemented using AnimationController and Tween in combination with AnimatedBuilder.
  - Creates a smooth vertical bouncing effect for the user avatar on the dashboard.
  - Provides visual feedback that the avatar is interactive, indicating that it can be tapped to edit the profile.
* **Option Cards Fade and Scale Animation:**
  - Implemented in the _buildOption() method using TweenAnimationBuilder.
  - Option cards scale from 0.8 to 1.0 and fade in with opacity animation, giving a smooth entrance effect.
  - Each card can have a slight delay (delay parameter) for a staggered animation effect.
* **Implicit Animations for UI Feedback:**
  - CircularProgressIndicator in login and registration screens provides a smooth, rotating animation during asynchronous operations.
  - While not manually animated, the progress indicators give a visual cue for loading states.
* **Navigation Transitions:**

- Default MaterialPageRoute transitions are used for screen navigation.
- Provides smooth sliding animations when moving between screens such as Login → Dashboard or Dashboard → Home/Leaderboard-/Edit Profile.

Animations:

```
class _ResultScreenState extends State<ResultScreen>
    with SingleTickerProviderStateMixin {
  late ConfettiController _confettiController;
  late AnimationController _controller;
  late Animation<double> _fade;
  late Animation<double> _scale;
  late Animation<Offset> _contentSlide;
  late Animation<Offset> _buttonSlide;

  String _message = '';
  Color _messageColor = Colors.grey;
  bool _shouldPlayConfetti = false;
  @override
  void initState() {
    super.initState();
    _confettiController = ConfettiController(
      duration: const Duration(seconds: 3),
    );
    _controller = AnimationController(
      vsync: this,
      duration: const Duration(milliseconds: 900),
    );
    _fade = Tween<double>(
      begin: 0.0,
      end: 1.0,
    ).animate(
      CurvedAnimation(
        parent: _controller,
        curve: Curves.easeIn,
      ),
    );
    _scale = Tween<double>(
      begin: 0.92,
      end: 1.0,
    ).animate(
      CurvedAnimation(
        parent: _controller,
        curve: Curves.elasticOut,
      ),
    );
    _contentSlide = Tween<Offset>(
      begin: const Offset(0, 0.15),
      end: Offset.zero,
    ).animate(
      CurvedAnimation(
        parent: _controller,
        curve: Curves.easeOut,
      ),
    );
```

```
      _buttonSlide = Tween<Offset>(
        begin: const Offset(0, 0.35),
        end: Offset.zero,
      ).animate(
        CurvedAnimation(
          parent: _controller,
          curve: const Interval(
            0.4,
            1.0,
            curve: Curves.easeOut,
          ),
        ),
      );

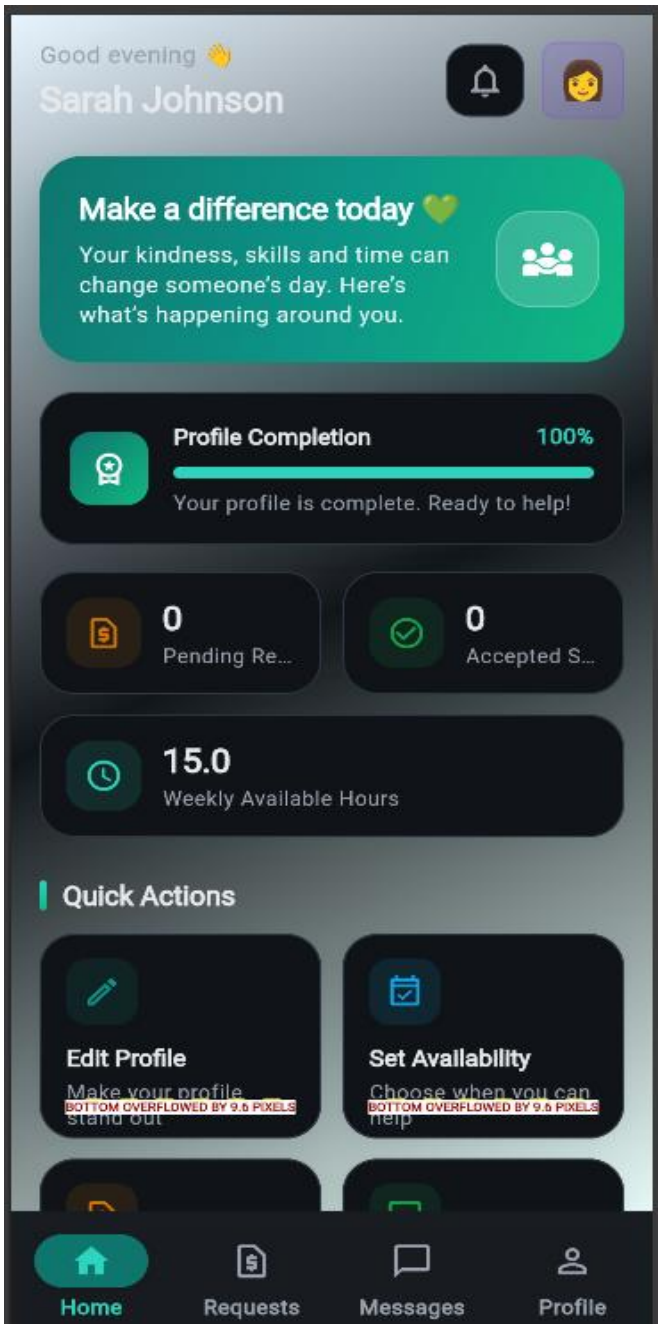
      // Volunteer session request result message
      _message = 'Your volunteer session request was submitted!';
      _messageColor = Colors.green;
      _shouldPlayConfetti = true;

      _controller.forward();

      if (_shouldPlayConfetti) {
        _confettiController.play();
      }
    }

    @override
    void dispose() {
      _confettiController.dispose();
      _controller.dispose();
      super.dispose();
    }
  }
```

Output:



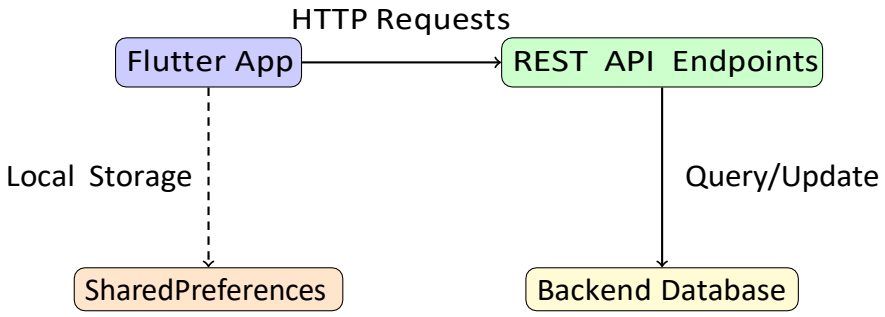
Result Screen

17 REST API Integration

Objective: Fetch data from API and display it.

Relation to Project: Used in leaderboard to fetch user scores.

```
final response = await http.get(Uri.parse('https://api.
    example.com/leaderboard'));
```



18 Testing and Debugging

Objective: To perform UI tests, unit tests, and use debugging tools to ensure the app functions correctly.

Explanation:

Testing is the process of verifying that the app works as intended. In this Flutter quiz app project, testing ensures that users can log in, attempt quizzes, view leaderboards, and edit profiles without errors. Types of testing include:

* **Unit Testing:** Tests individual functions or classes.

  Example: Checking if AuthService.login() returns a valid token.

* **Widget Testing:** Verifies that UI components display correctly and respond to user interactions.

  Example: Checking if the LoginScreen shows a "Login" button.

* **Integration Testing:** Tests end-to-end flow, including REST API calls.

  Example: Login → Fetch Quiz → Submit Answers → View Leaderboard.

* **Manual Testing:** Running the app on a device or emulator to test user interactions directly.

Debugging involves identifying and fixing errors in UI, API calls, or local storage. Tools used include Flutter DevTools, print/log statements, try/catch blocks, and Postman for API testing.

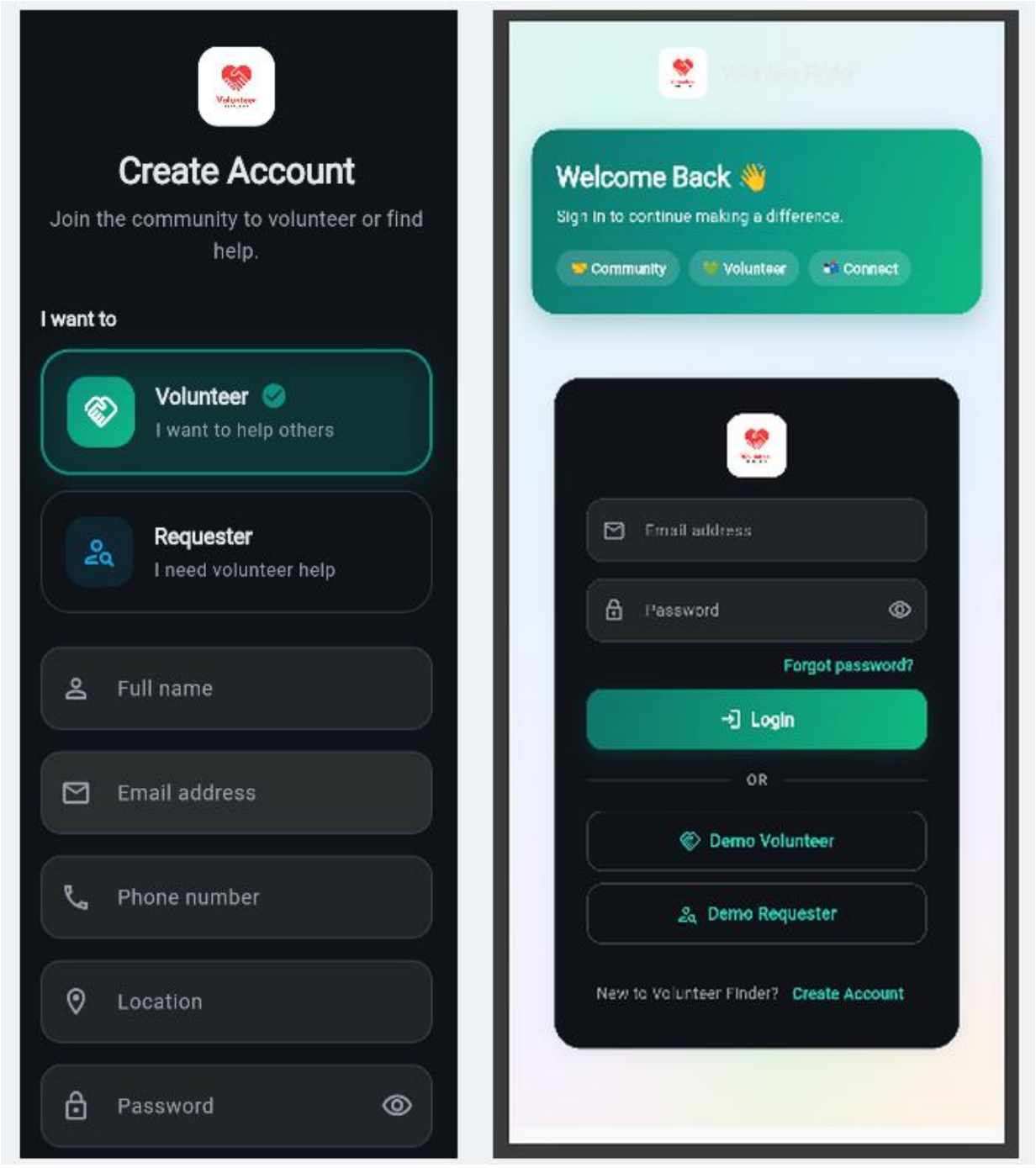
Sample UI Test Code:

```
testWidgets('Login Screen loads', (tester) async {
  await tester.pumpWidget(MyApp());
  expect(find.text('Login'), findsOneWidget);
});
```

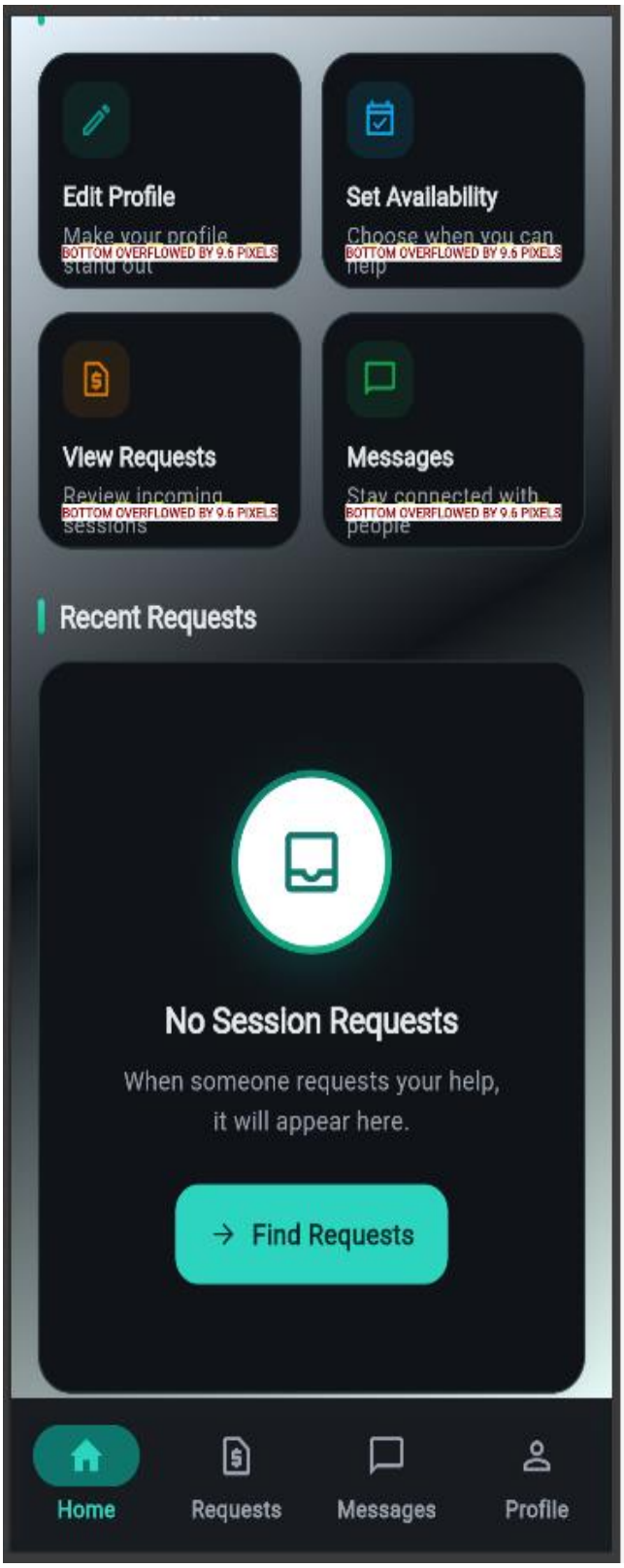
Sample Debugging Example:

```
try {
  final token = await authService.login(email, password);
  if (token == null) {
    print('Login failed: Invalid response');
  }
} catch (e) {
  print('Login error: $e');
}
```

19 Work Flow Of Screens:



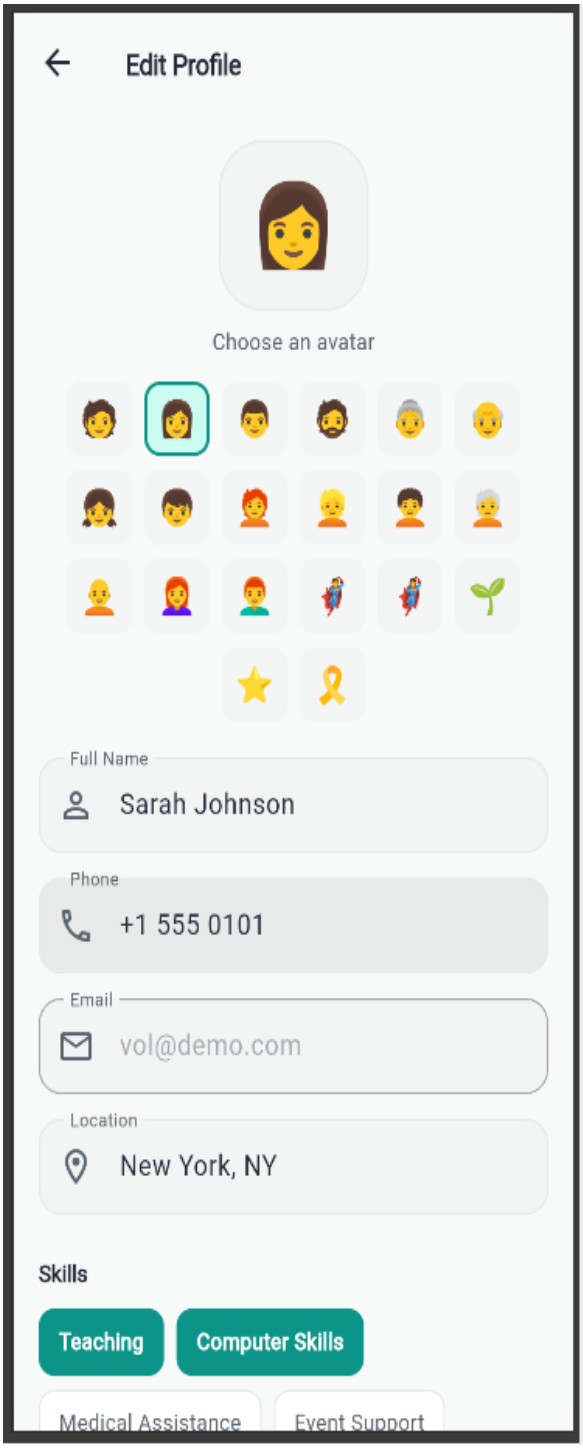
Register Screen **Register Screen**



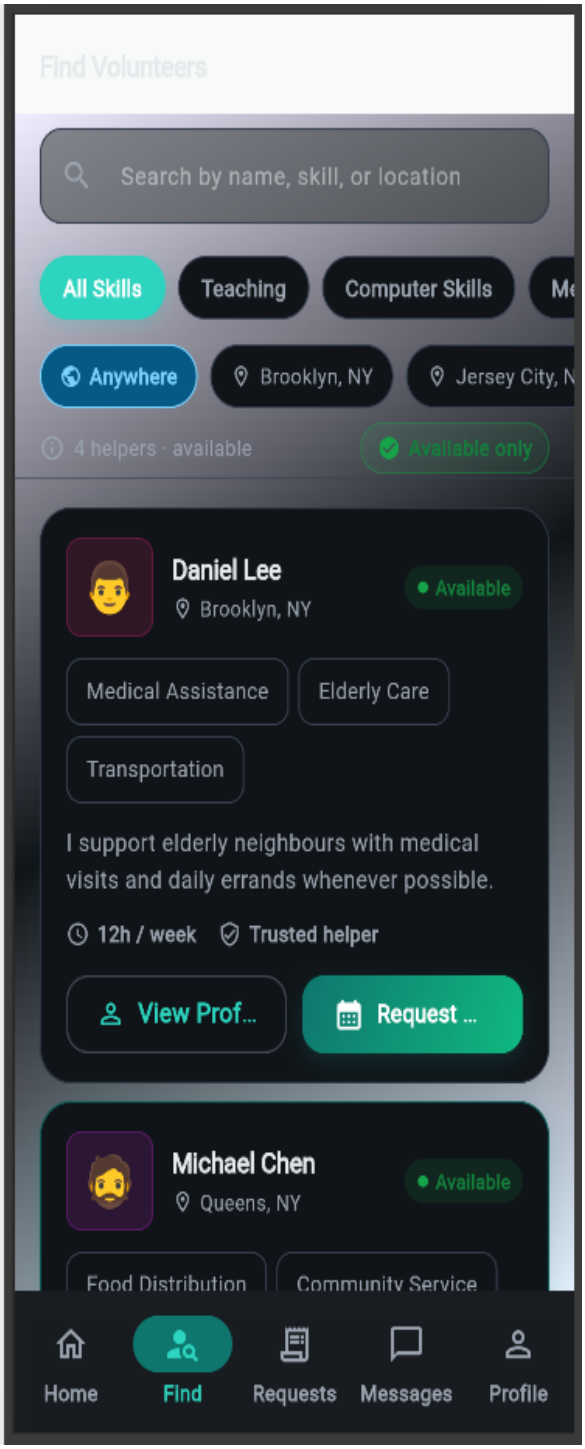
Home Screen



Profile Screen



Profile edit Screen



Job find Screen

20 Conclusion

Through these experiments, we explored core Flutter principles such as widgets, layouts, navigation, animations, and state management. **The Volunteer Finder App** effectively demonstrates the integration of these concepts in a complete mobile application. It showcases how Flutter's declarative UI and reactive framework can be used to build dynamic, responsive, and visually appealing applications..

Additionally, the project provided hands-on experience with user authentication, volunteer profile management, searching and filtering volunteers, session requests, and Firebase integration. The use of Material Design components and custom themes ensured a consistent and engaging user interface. Overall, this project not only strengthened our technical skills in Flutter and Dart but also improved our ability to design and develop scalable mobile applications efficiently.